

# Logic Tensor Networks

Samy Badreddine<sup>a,b,\*</sup>, Artur d'Ávila Garcez<sup>c</sup>, Luciano Serafini<sup>d</sup>, Michael Spranger<sup>a,b</sup>

<sup>a</sup>Sony Computer Science Laboratories Inc, 3-14-13 Higashigotanda, 141-0022, Tokyo, Japan

<sup>b</sup>Sony AI Inc, 1-7-1 Konan, 108-0075, Tokyo, Japan

<sup>c</sup>City, University of London, Northampton Square, EC1V 0HB, London, United Kingdom

<sup>d</sup>Fondazione Bruno Kessler, Via Sommarive 18, 38123, Trento, Italy

---

## Abstract

Attempts at combining logic and neural networks into neurosymbolic approaches have been on the increase in recent years. In a neurosymbolic system, symbolic knowledge assists deep learning, which typically uses a sub-symbolic distributed representation, to learn and reason at a higher level of abstraction. We present Logic Tensor Networks (LTN), a neurosymbolic framework that supports querying, learning and reasoning with both rich data and abstract knowledge about the world. LTN introduces a fully differentiable logical language, called Real Logic, whereby the elements of a first-order logic signature are grounded onto data using neural computational graphs and first-order fuzzy logic semantics. We show that LTN provides a uniform language to represent and compute efficiently many of the most important AI tasks such as multi-label classification, relational learning, data clustering, semi-supervised learning, regression, embedding learning and query answering. We implement and illustrate each of the above tasks with several simple explanatory examples using TensorFlow 2. The results indicate that LTN can be a general and powerful framework for neurosymbolic AI.

*Keywords:* Neurosymbolic AI, Deep Learning and Reasoning, Many-valued Logics.

---

## 1. Introduction

Artificial Intelligence (AI) agents are required to learn from their surroundings and reason about what has been learned to make decisions, act in the world, or react to various stimuli. The latest Machine Learning (ML) has adopted mostly a pure sub-symbolic learning approach. Using distributed representations of entities, the latest ML performs quick decision-making without building a comprehensible model of the world. While achieving impressive results in computer vision, natural language, game playing, and multimodal learning, such approaches are known to be data inefficient and to struggle at out-of-distribution generalization. Although the use of appropriate inductive biases can alleviate such shortcomings, in general, sub-symbolic models lack comprehensibility. By contrast, symbolic AI is based on rich, high-level representations of the world that use human-readable symbols. By *rich knowledge*, we refer to logical representations which are

---

\*Corresponding author

Email addresses: badreddine.samy@gmail.com (Samy Badreddine), a.garcez@city.ac.uk (Artur d'Ávila Garcez), serafini@fbk.eu (Luciano Serafini), michael.spranger@sony.com (Michael Spranger)

more expressive than propositional logic or propositional probabilistic approaches, and which can express knowledge using full first-order logic, including universal and existential quantification ( $\forall x$  and  $\exists y$ ), arbitrary  $n$ -ary relations over variables, e.g.  $R(x, y, z, \dots)$ , and function symbols, e.g.  $\text{fatherOf}(x)$ ,  $x + y$ , etc. Symbolic AI has achieved success at theorem proving, logical inference, and verification. However, it also has shortcomings when dealing with incomplete knowledge. It can be inefficient with large amounts of inaccurate data and lack robustness to outliers. Purely symbolic decision algorithms usually have high computational complexity making them impractical for the real world. It is now clear that the predominant approach to ML, where learning is based on recognizing the latent structures hidden in the data, is insufficient and may benefit from symbolic AI [17]. In this context, neurosymbolic AI, which stems from *neural* networks and *symbolic* AI, attempts to combine the strength of both paradigms (see [16, 40, 54] for recent surveys). That is to say, combine reasoning with complex representations of knowledge (knowledge-bases, semantic networks, ontologies, trees, and graphs) with learning from complex data (images, time series, sensorimotor data, natural language). Consequently, a main challenge for neurosymbolic AI is the grounding of symbols, including constants, functional and relational symbols, into real data, which is akin to the longstanding *symbol grounding* problem [30].

Logic Tensor Networks (LTN) are a neurosymbolic framework and computational model that supports learning and reasoning about data with rich knowledge. In LTN, one can represent and effectively compute the most important tasks of deep learning with a fully differentiable first-order logic language, called Real Logic, which adopts infinitely many truth-values in the interval  $[0,1]$  [22, 25]. In particular, LTN supports the specification and computation of the following AI tasks uniformly using the same language: data clustering, classification, relational learning, query answering, semi-supervised learning, regression, and embedding learning.

LTN and Real Logic were first introduced in [62]. Since then, LTN has been applied to different AI tasks involving perception, learning, and reasoning about relational knowledge. In [18, 19], LTN was applied to semantic image interpretation whereby relational knowledge about objects was injected into deep networks for object relationship detection. In [6], LTN was evaluated on its capacity to perform reasoning about ontological knowledge. Furthermore, [7] shows how LTN can be used to learn an embedding of concepts into a latent real space by taking into consideration ontological knowledge about such concepts. In [3], LTN is used to annotate a reinforcement learning environment with prior knowledge and incorporate latent information into an agent. In [42], authors embed LTN in a state-of-the-art convolutional object detector. Extensions and generalizations of LTN have also been proposed in the past years, such as LYRICS [47] and *Differentiable Fuzzy Logic* (DFL) [68, 69]. LYRICS provides an input language allowing one to define background knowledge using a first-order logic where predicate and function symbols are grounded onto any computational graph. DFL analyzes how a large collection of fuzzy logic operators behave in a differentiable learning setting. DFL also introduces new semantics for fuzzy logic implications called sigmoidal implications, and it shows that such semantics outperform other semantics in several semi-supervised machine learning tasks.

This paper provides a thorough description of the full formalism and several extensions of LTN. We show using an extensive set of explanatory examples, how LTN can be applied to solve many ML tasks with the help of logical knowledge. In particular, the earlier versions of LTN have been extended with: (1) *Explicit domain declaration*: constants, variables, functions and predicates are now domain typed (e.g. the constants *John* and *Paris* can be from the domain of *person* and *city*, respectively). The definition of structured domains is also possible (e.g. the domain *couple* can be defined as the Cartesian product of two domains of persons); (2) *Guarded quantifiers*: guarded

universal and existential quantifiers now allow the user to limit the quantification to the elements that satisfy some Boolean condition, e.g.  $\forall x : \text{age}(x) < 10 (\text{playsPiano}(x) \rightarrow \text{enfantProdige}(x))$  restricts the quantification to the cases where *age* is lower than 10; (3) *Diagonal quantification*: Diagonal quantification allows the user to write statements about specific tuples extracted in order from  $n$  variables. For example, if the variables *capital* and *country* both have  $k$  instances such that the  $i$ -th instance of *capital* corresponds to the  $i$ -th instance of *country*, one can write  $\forall \text{Diag}(\text{capital}, \text{country}) \text{capitalOf}(\text{capital}, \text{country})$ .

Inspired by the work of [69], this paper also extends the product t-norm configuration of LTN with the generalized mean aggregator, and it introduces solutions to the vanishing or exploding gradient problems. Finally, the paper formally defines a semantic approach to *refutation-based reasoning* in Real Logic to verify if a statement is a logical consequence of a knowledge base. Example 4.8 proves that this new approach can better capture logical consequences compared to simply querying unknown formulas after learning (as done in [6]).

The new version of LTN has been implemented in TensorFlow 2 [1]. Both the LTN library and the code for the examples used in this paper are available at <https://github.com/logictensornetworks/logictensornetworks>.

The remainder of the paper is organized as follows: In Section 2, we define and illustrate Real Logic as a fully-differentiable first-order logic. In Section 3, we specify learning and reasoning in Real Logic and its modeling into deep networks with Logic Tensor Networks (LTN). In Section 4, we illustrate the reach of LTN by investigating a range of learning problems from clustering to embedding learning. In Section 5, we place LTN in the context of the latest related work in neurosymbolic AI. In Section 6 we conclude and discuss directions for future work. The Appendix contains information about the implementation of LTN in TensorFlow 2, experimental set-ups, the different options for the differentiable logic operators, and a study of their relationship with gradient computations.

## 2. Real Logic

### 2.1. Syntax

Real Logic forms the basis of Logic Tensor Networks. Real Logic is defined on a first-order language  $\mathcal{L}$  with a signature that contains a set  $\mathcal{C}$  of constant symbols (objects), a set  $\mathcal{F}$  of functional symbols, a set  $\mathcal{P}$  of relational symbols (predicates), and a set  $\mathcal{X}$  of variable symbols.  $\mathcal{L}$ -formulas allow us to specify relational knowledge with variables, e.g. the atomic formula  $\text{is\_friend}(v_1, v_2)$  may state that the person  $v_1$  is a friend of the person  $v_2$ , the formula  $\forall x \forall y (\text{is\_friend}(x, y) \rightarrow \text{is\_friend}(y, x))$  states that the relation *is\_friend* is symmetric, and the formula  $\forall x (\exists y (\text{Italian}(x) \wedge \text{is\_friend}(x, y)))$  states that every person has a friend that is Italian. Since we are interested in learning and reasoning in real-world scenarios where degrees of truth are often fuzzy and exceptions are present, formulas can be partially true, and therefore we adopt fuzzy semantics.

Objects can be of different types. Similarly, functions and predicates are typed. Therefore, we assume there exists a non-empty set of symbols  $\mathcal{D}$  called *domain symbols*. To assign types to the elements of  $\mathcal{L}$  we introduce the functions  $\mathbf{D}$ ,  $\mathbf{D}_{\text{in}}$  and  $\mathbf{D}_{\text{out}}$  such that:

- $\mathbf{D} : \mathcal{X} \cup \mathcal{C} \rightarrow \mathcal{D}$ . Intuitively,  $\mathbf{D}(x)$  and  $\mathbf{D}(c)$  returns the domain of a variable  $x$  or a constant  $c$ .
- $\mathbf{D}_{\text{in}} : \mathcal{F} \cup \mathcal{P} \rightarrow \mathcal{D}^*$ , where  $\mathcal{D}^*$  is the Kleene star of  $\mathcal{D}$ , that is the set of all finite sequences of symbols in  $\mathcal{D}$ . Intuitively,  $\mathbf{D}_{\text{in}}(f)$  and  $\mathbf{D}_{\text{in}}(p)$  returns the domains of the arguments of a